Acknowledgement

We would like to express our sincere gratitude to our teacher and mentor for their guidance, encouragement, and feedback while we completed this case study on the **Inventory Management System**. Their support helped us understand and apply the concepts of **Management Information System (MIS)** in a practical way.

We also thank our families for their steady encouragement, our friends and classmates for their helpful criticism, and every member of our group for the cooperation and teamwork that made this project possible.

Introduction

Background

Any business that sells or uses physical goods needs to know what it has, where it belongs and when to buy more. Many small organizations still track stock in notebooks or spreadsheets, which makes it slow to find a product, easy to miscount and hard to see the overall value of what is on the shelf. An inventory management system replaces these manual records with one shared database and a simple web interface, so that the whole team works from the same up-to-date numbers.

Problem Definition

With manual or spreadsheet-based tracking, staff often cannot tell which products are running low until they run out. Prices, quantities and categories are scattered across files, duplicate entries appear, and there is no quick way to see total stock or the value of the inventory. Access is also uncontrolled: anyone with the file can change anything. The business needs a system that stores products and categories in a structured way, flags low stock automatically, summarizes the key figures on a dashboard and separates what administrators and ordinary staff are allowed to do.

Objectives

The main objective is to design and develop a web-based Inventory Management System that helps administrators and staff keep accurate stock records. Specifically, the system aims to:

- Store products with their purchase price, selling price, quantity and minimum stock level, grouped by category.
- Show a status for every product (in stock, low stock or out of stock) so that shortages are noticed early.
- Present a dashboard with total products, total categories, total stock, the number of low-stock products, the total inventory value and the stock held in each category.
- Provide role-based access for two kinds of users, admin and staff, with a staff list visible to administrators.

Scope

The system covers the day-to-day stock records of a single business. It includes user login, a dashboard, product management (list, create and edit), category management (list, create and edit) and a staff list. Lists are paginated at 20 records per page. The scope does not include sales, billing, purchase orders, supplier records or barcode scanning. These can be added later because the database and interface are built in separate, extendable parts.

Planning

Planning sets the goals, priorities and resources of a project before any code is written. For this system it meant deciding which records to keep, who may change them and which pages are needed, and it helped us spot risks early. We assessed five kinds of feasibility.

Assessing Feasibility

- **Economic feasibility:** Development cost is low because the system relies on open-source tools and inexpensive hosting. It replaces paper and spreadsheet work, so the main return is saved staff time and fewer stock losses from running out of popular items or over-buying slow ones.
- **Technical feasibility:** The system needs only common web technologies: a backend that serves data, a relational database such as MySQL, and a component-based frontend with one page per screen. These tools are well documented and within the skills of a student development team.
- **Operational feasibility:** The interface is a small set of familiar list and form pages, so staff need little training. Administrators gain a quick overview from the dashboard, and staff can focus on updating products and categories.
- **Schedule feasibility:** Using an agile approach, the work splits into short sprints: database design, backend logic, frontend pages and testing. A working version with the dashboard, products, categories and staff list can be finished within an academic semester.
- **Legal feasibility:** The system stores only business stock data and staff login details. Passwords must be stored hashed, access is limited by role, and personal data such as staff emails is kept private. No third-party copyrighted data is used.

System Analysis

System analysis breaks the problem into parts that can be understood and checked against what users need. It describes what the system must do (functional requirements) and how well it must do it (non-functional requirements).

Functional Requirements

1. **Authentication and roles**
    - Users must log in with an email and password.
    - Each user has a role, either admin or staff, which decides what they can access.
2. **Dashboard**
    - Show total products, total categories and total stock.
    - Show the number of low-stock products, where quantity has reached the product's minimum stock.
    - Show the inventory value, the total value of all products in stock.
    - Show the total stock of each category.
3. **Product management**
    - List products 20 per page with id, name, description, purchase price, selling price, quantity, category and a status of in stock, low stock or out of stock.
    - Create a product with name, description, purchase price, selling price, quantity, minimum stock and a category chosen from a dropdown.
    - Edit an existing product using the same fields.
4. **Category management**
    - List categories 20 per page with id, name, description and the number of products in each.
    - Create and edit categories with a name and description.
5. **Staff management**
    - List staff members 20 per page with id, name, email and role.

Non-Functional Requirements

1. **Performance:** pages and lists should load within 2 to 3 seconds, which is why lists are paginated.

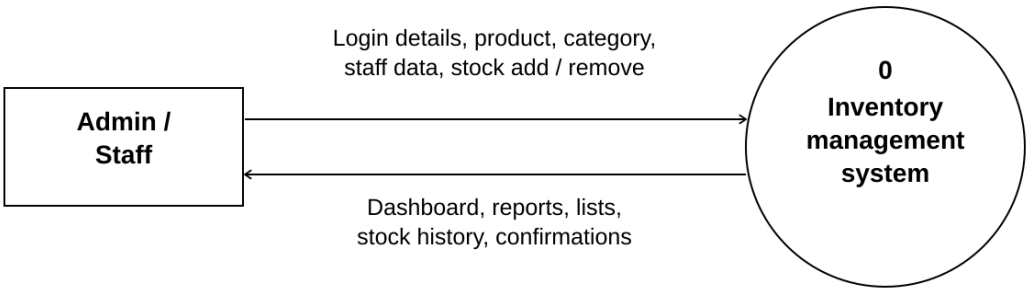
2. **Usability:** the interface should be simple enough for staff with limited computer experience.
3. **Security:** passwords are stored hashed, emails are unique, and pages are available only to the roles allowed to use them.
4. **Reliability:** the system should be available during working hours, and stock figures must stay consistent with the stored records.
5. **Accuracy:** prices are stored as decimals with two places, so inventory value is calculated without rounding errors.
6. **Scalability:** the database should handle a growing number of products, categories and users.
7. **Platform compatibility:** the web application should work on all major browsers and on mobile screens.

Data Flow Diagram (DFD)

A data flow diagram shows how data moves between the people who use the system, the processes that handle it and the stores that keep it. Three levels are drawn, each adding detail to the one before.

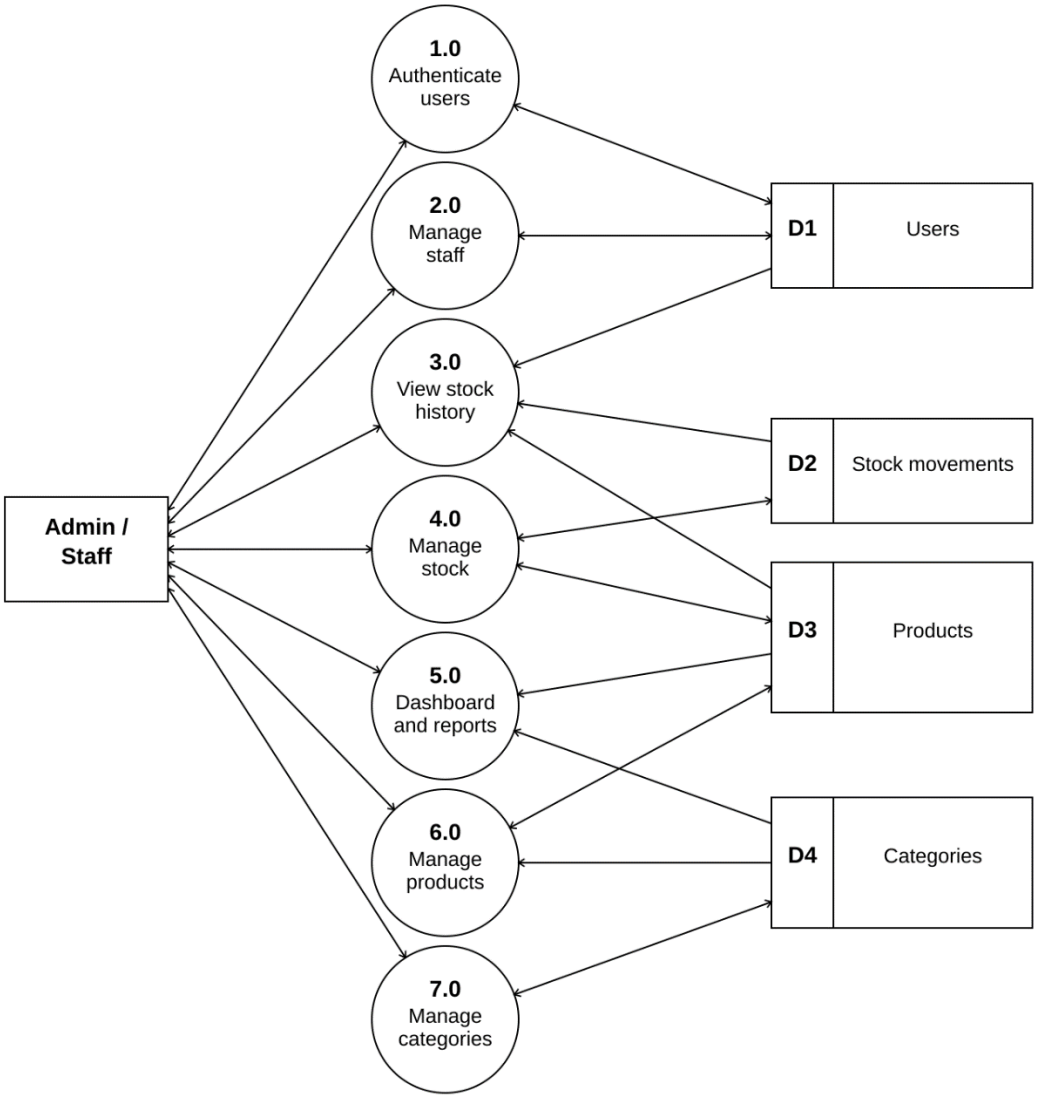
Level 0 DFD

The context diagram treats the whole system as one process. The only external entity is the logged-in user, either an admin or a member of staff.



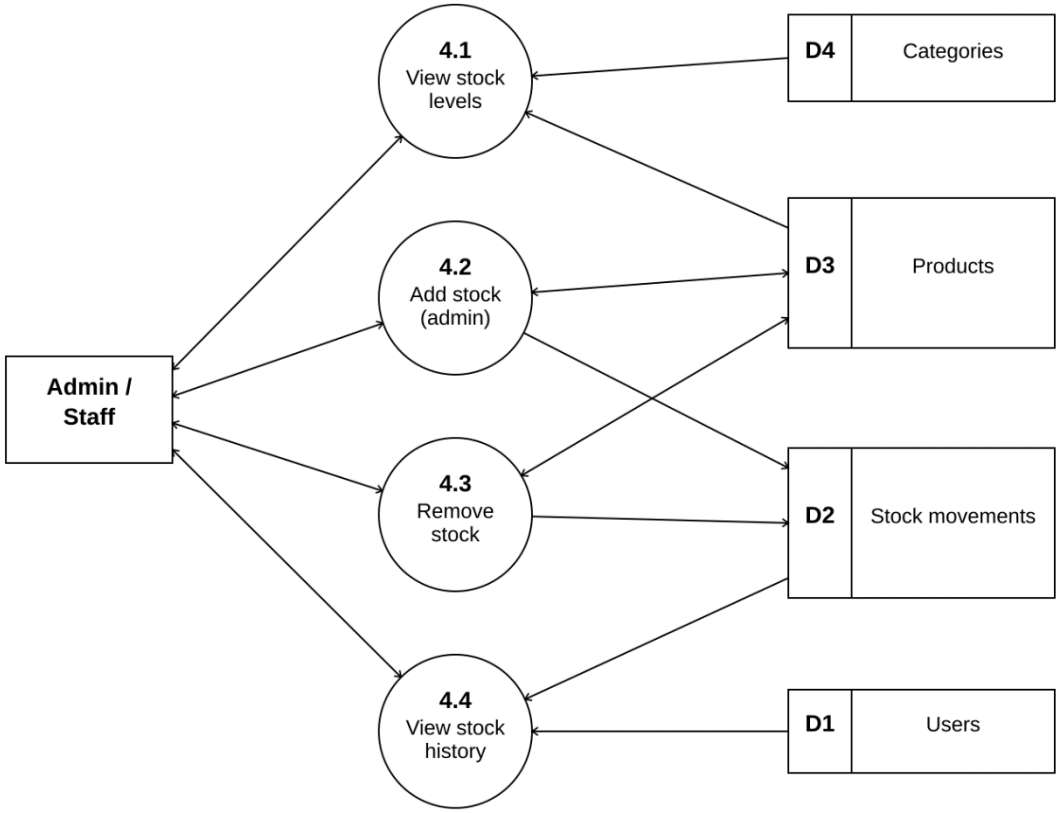
Level 1 DFD

Level 1 splits the system into five processes: authenticate users, manage staff, manage categories, view the dashboard and manage products. They read and write three data stores: Users (D1), Categories (D2) and Products (D3). The dashboard only reads from Categories and Products, and it calculates its totals from those records.



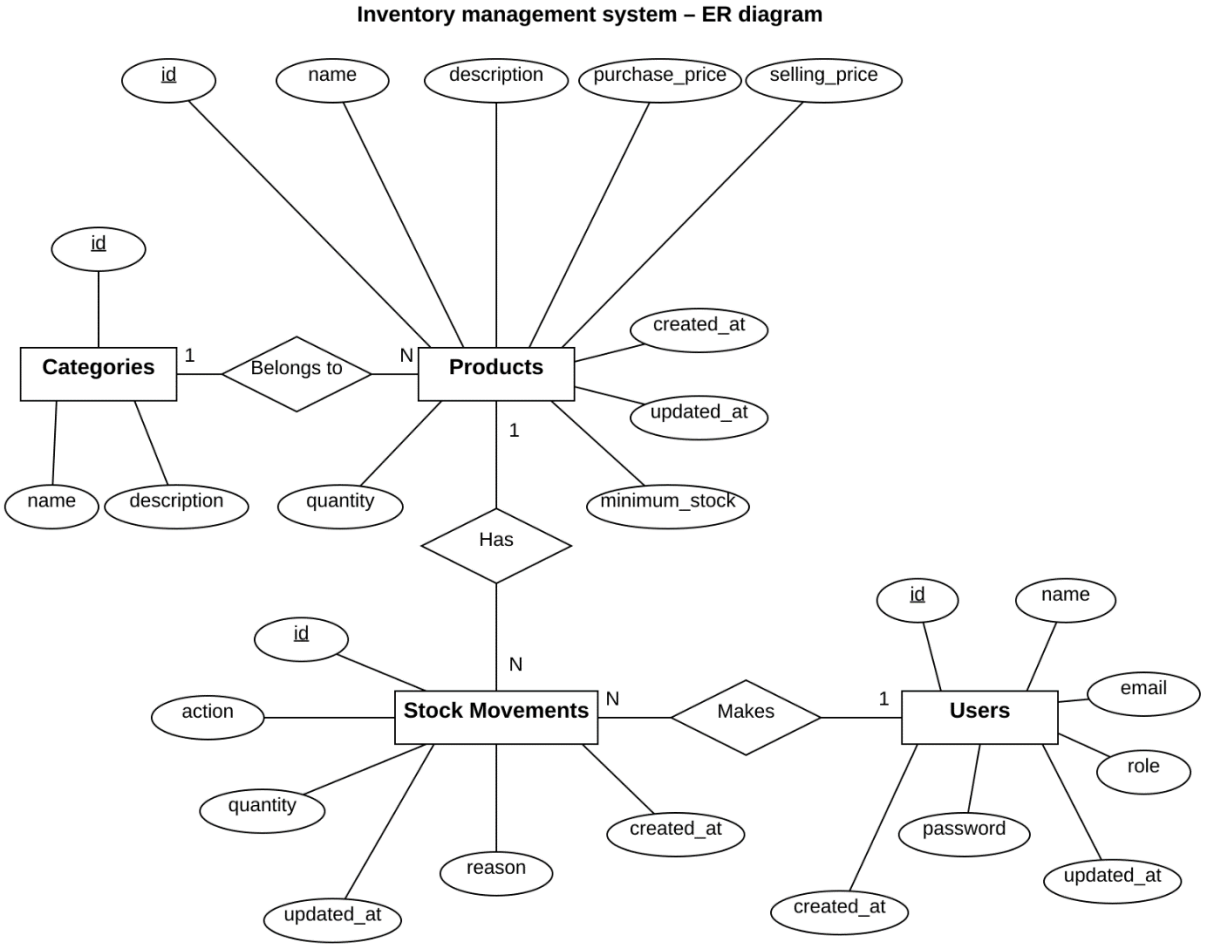
Level 2 DFD

Level 2 expands process 5.0, Manage products, into three sub-processes: view the product list (5.1), create a product (5.2) and update a product (5.3). Creating and updating also read the Categories store so that every product is linked to a valid category.



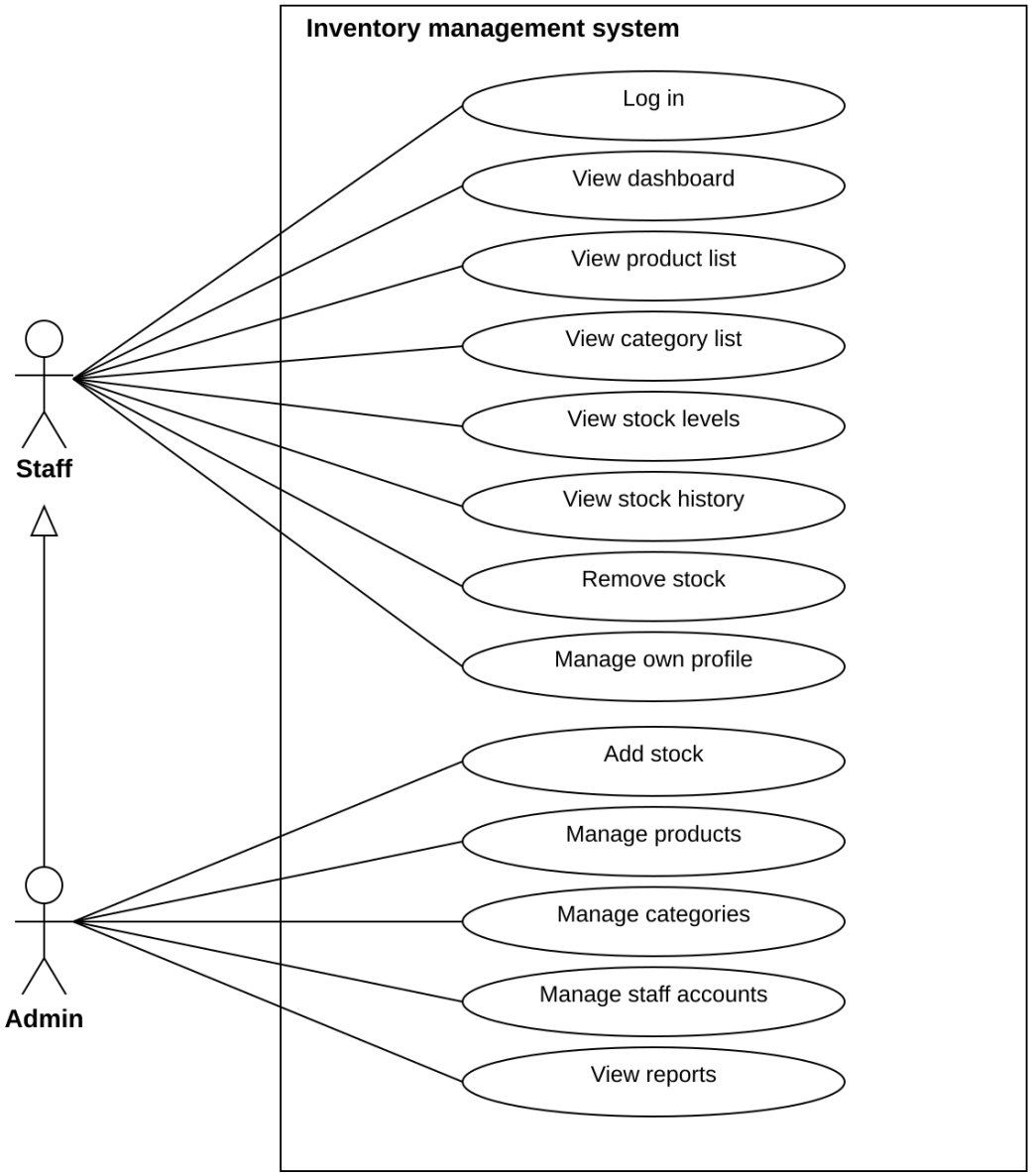
Entity Relationship Diagram (ER Diagram)

The ER diagram uses Chen notation: rectangles are entities, ovals are attributes (a primary key is underlined) and diamonds are relationships with their cardinality.



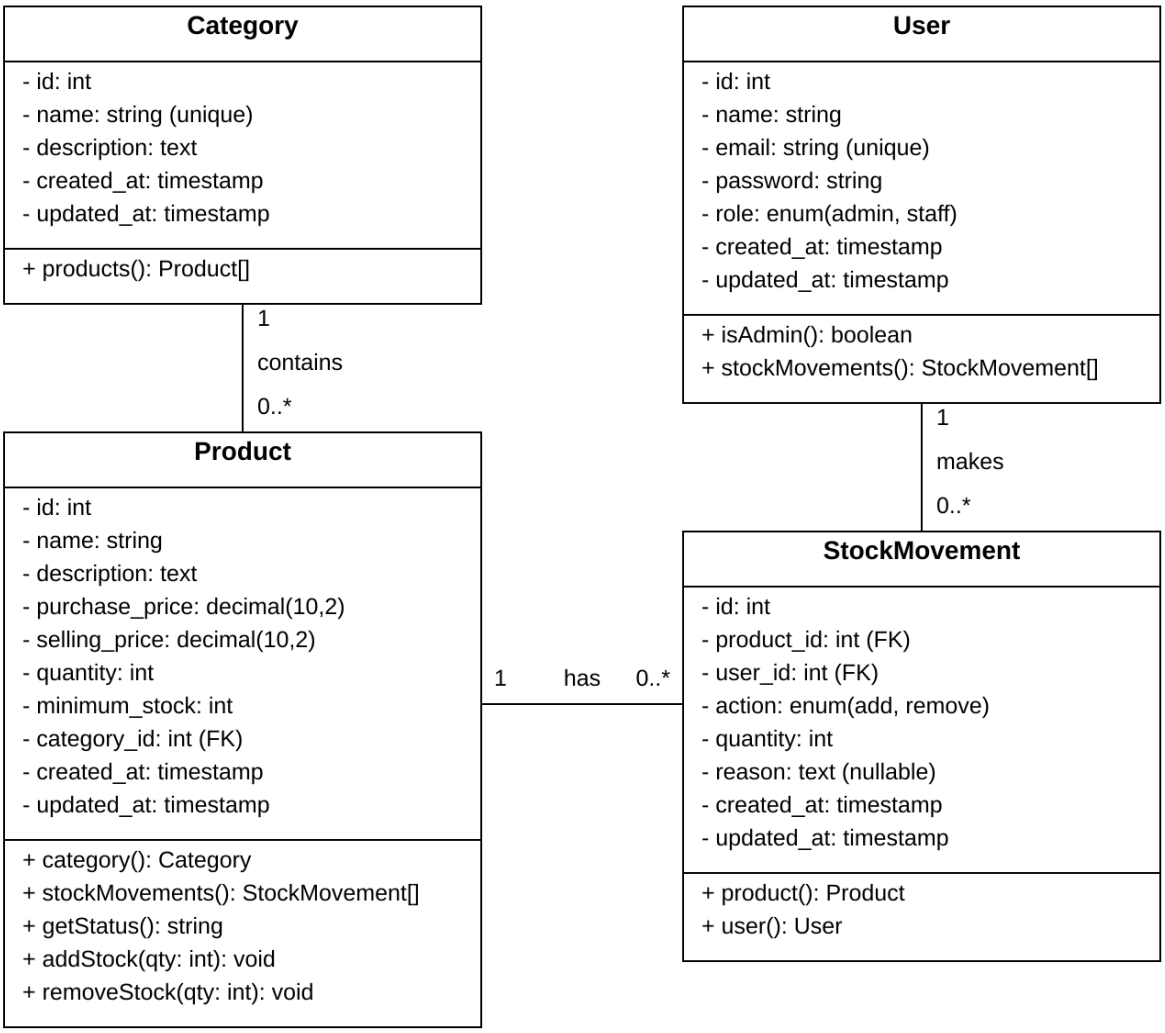
Use Case Diagram

Staff can log in, view the dashboard, and view, create and edit products and categories. Admin inherits everything Staff can do and can also view the staff list.

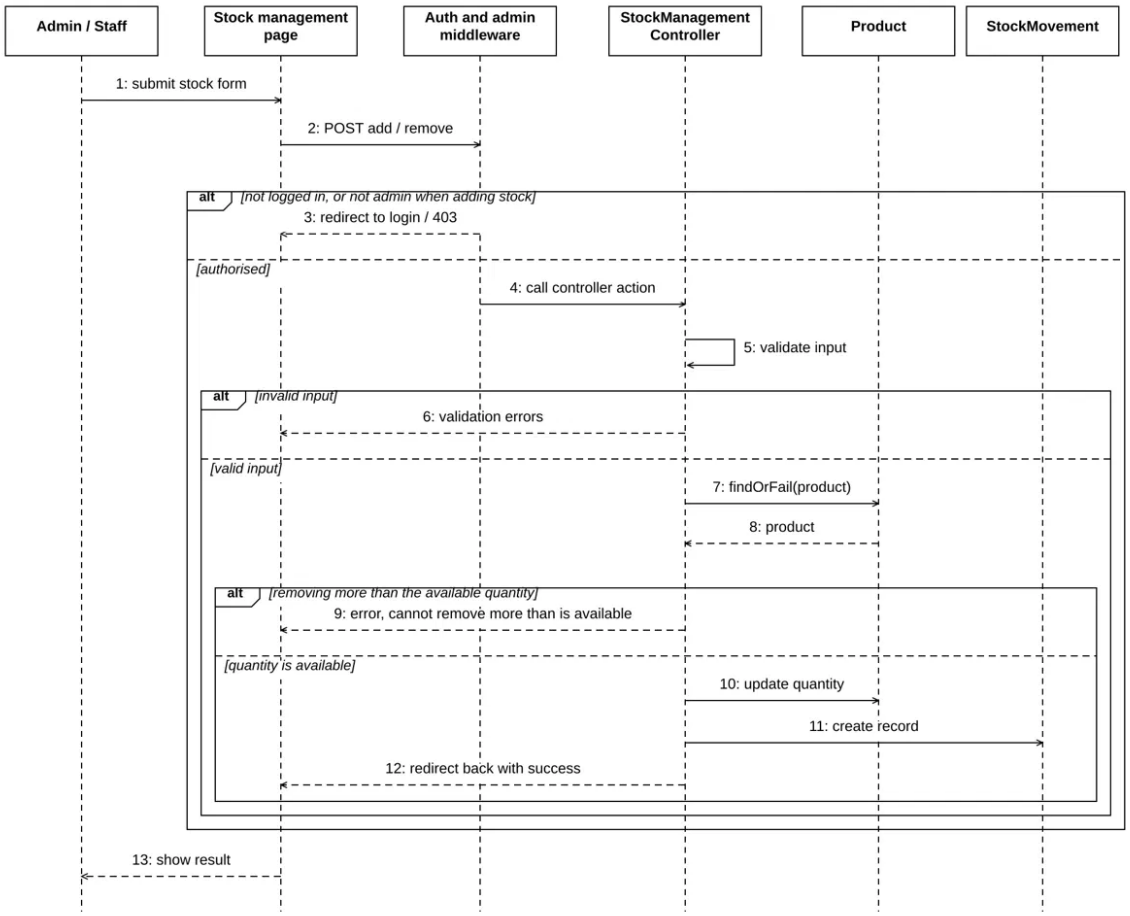


Class Diagram

The class diagram shows the main classes and their attributes. A Category contains zero or more Products. A User has a Role, which is either admin or staff. The Dashboard class represents the calculated figures and uses Product and Category. Methods such as `getStatus()` and `isLowStock()` are suggested operations based on the status shown on the product list.



Sequence Diagram



System Design

System design turns the analysis into a blueprint for building the system. A well-planned design keeps the database, the backend and the interface working together and makes the system easier to maintain and extend.

Database Design

The database is organized into three tables. `categories` groups products, `products` holds the stock records and `users` holds the people who log in. Each product belongs to exactly one category through the foreign key `category_id`, while a category can hold many products. The `users` table is independent of the other two, and its `role` column controls access.

The product status is not stored. It is worked out from two columns: a product with quantity 0 is out of stock, one with quantity at or below `minimum_stock` is low stock, and any other product is in stock. In the same way, the dashboard totals, the number of products in each category and the inventory value are calculated from the stored rows, which avoids duplicated data.

Table: products

Column	Type	Key	Description
id	INT, auto-increment	Primary key	Unique product number
name	VARCHAR(255)		Product name
description	TEXT		Product details
purchase_price	DECIMAL(10,2)		Cost price per unit
selling_price	DECIMAL(10,2)		Selling price per unit
quantity	INT		Units currently in stock
minimum_stock	INT		Level at which the product counts as low stock
category_id	INT	Foreign key	References categories.id
created_at, updated_at	TIMESTAMP		Record dates

Table: users

Column	Type	Key	Description
id	INT, auto-increment	Primary key	Unique user number
name	VARCHAR(255)		Full name
email	VARCHAR(255)	Unique	Login email
password	VARCHAR(255)		Hashed password
role	ENUM('admin','staff')		Access level
created_at, updated_at	TIMESTAMP		Record dates

Table: categories

Column	Type	Key	Description
id	INT, auto-increment	Primary key	Unique category number
name	VARCHAR(255)		Category name
description	TEXT		Category details

Implementation of the ER Diagram into Physical Tables Using MySQL

```
CREATE TABLE categories (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    description TEXT
);

CREATE TABLE products (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    purchase_price DECIMAL(10, 2) NOT NULL,
    selling_price DECIMAL(10, 2) NOT NULL,
    quantity INT NOT NULL DEFAULT 0,
    minimum_stock INT NOT NULL DEFAULT 0,
    category_id INT NOT NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    FOREIGN KEY (category_id) REFERENCES categories(id)
);
```

```
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role ENUM('admin', 'staff') NOT NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL
);

CREATE TABLE stock_movements (
    id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT NOT NULL,
    user_id INT NOT NULL,
    action ENUM('add', 'remove') NOT NULL,
    quantity INT NOT NULL,
    reason TEXT,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,

    FOREIGN KEY (product_id) REFERENCES products(id),
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Designing Interfaces and Dialogues

Good interfaces make the system easy to learn and use. The Inventory Management System has eight pages. Each page receives its data from the backend and, where it has a form, sends the entered values back to be saved.

Page	Purpose	Data shown or sent
Dashboard	Give an overview of stock	Total products, total categories, total stock, low-stock count, inventory value, and the total stock of each category
Product / Index	List products, 20 per page	id, name, description, purchase price, selling price, quantity, status and category (id and name)
Product / Create	Add a product	Receives the category list (id, name) for a dropdown; sends name, description, purchase price, selling price, quantity, minimum stock and category id
Product / Edit	Change a product	Receives the product and the category list; sends the same fields as Create
Category / Index	List categories, 20 per page	id, name, description and number of products
Category / Create	Add a category	Sends name and description
Category / Edit	Change a category	Receives the category (id, name, description); sends the changed values
Staff / Index	List staff, 20 per page	id, name, email and role (admin or staff)

Interface Details

1. **Dashboard:** Five summary figures sit at the top. Below them, a list of categories shows how much stock each one holds, so the biggest and smallest categories can be compared at a glance.
2. **Product list:** Each row carries a coloured status label (in stock, low stock, out of stock) so that products needing attention stand out. Buttons lead to Create and Edit.

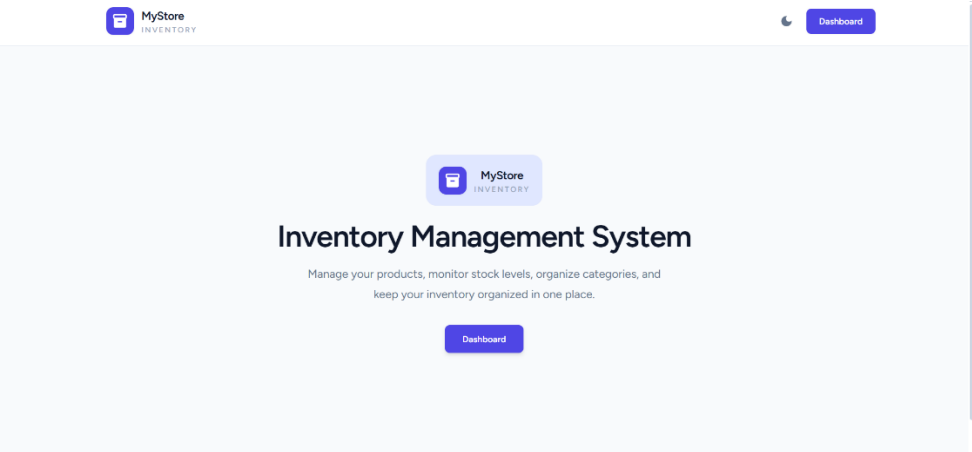
3. **Product forms:** Create and Edit share one layout. The category is chosen from a dropdown rather than typed, which prevents invalid category ids.
4. **Category pages:** The list shows a product count, and the forms ask only for a name and a description.
5. **Staff list:** A read-only table of users and their roles, intended for administrators.

Sample Dialogues

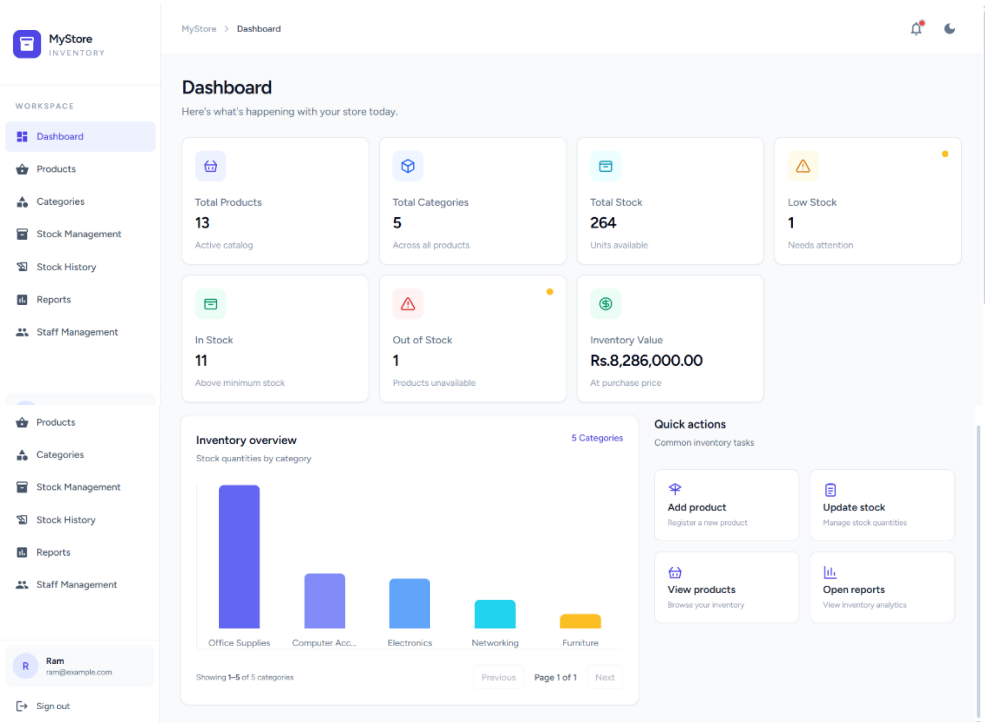
- Placeholder text: "Enter product name...", "Select a category..."
- Validation error: "Please enter a valid price and quantity."
- Confirmation: "Product saved successfully."
- Low-stock notice on the dashboard: "3 products are low on stock."
- Access message: "You do not have permission to view this page."

Application User Interface

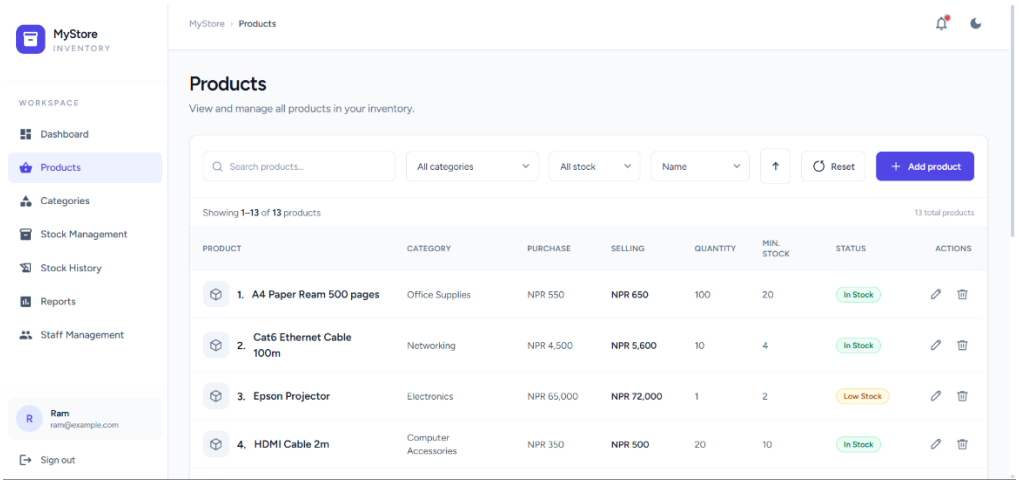
1. Home Page



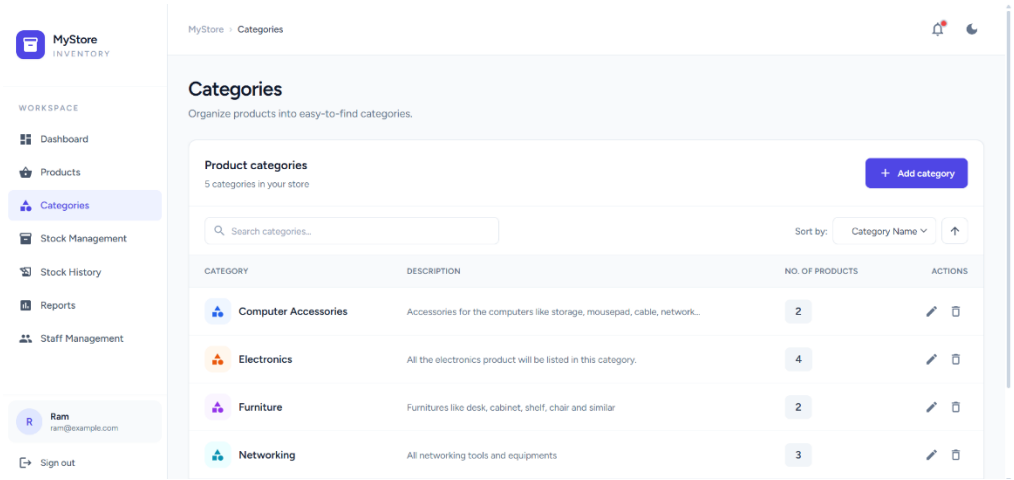
2. Dashboard Page



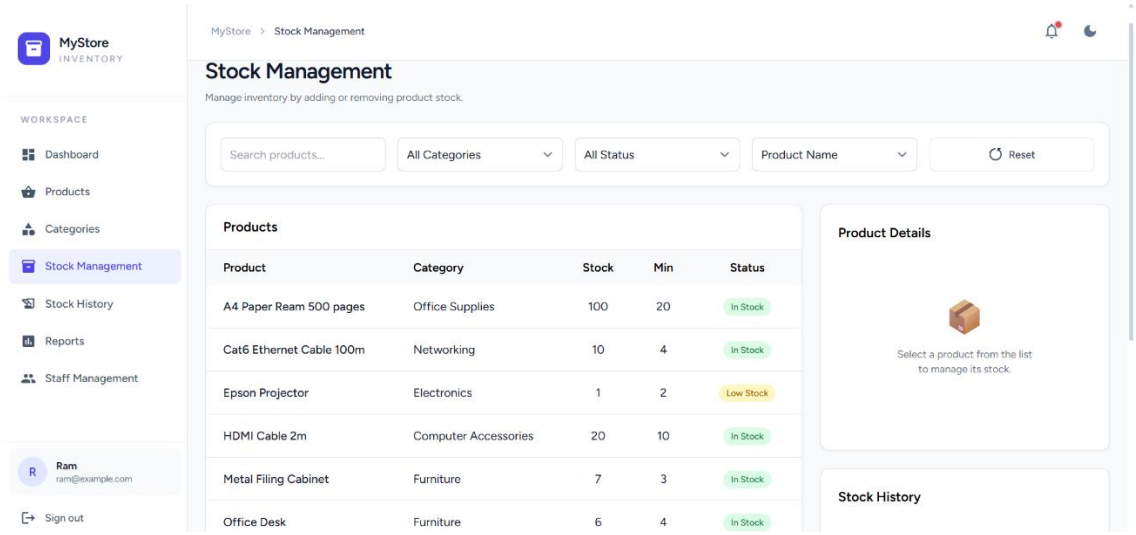
3. Products page



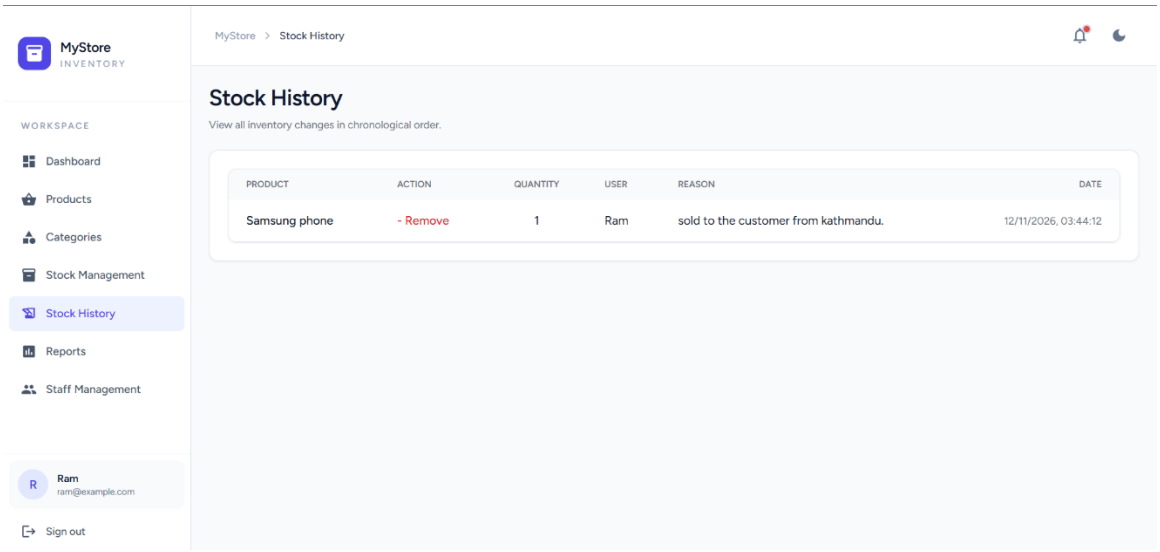
4. Categories page



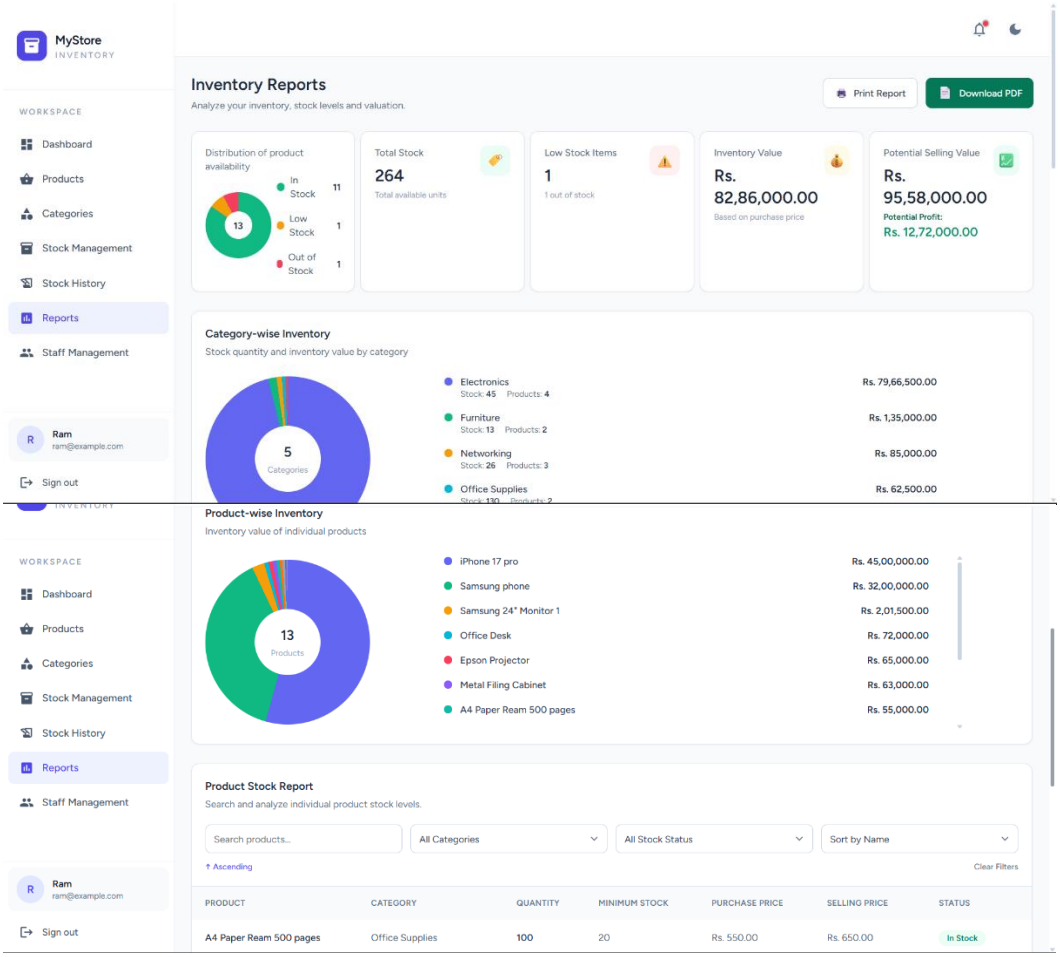
5. Stock Management page



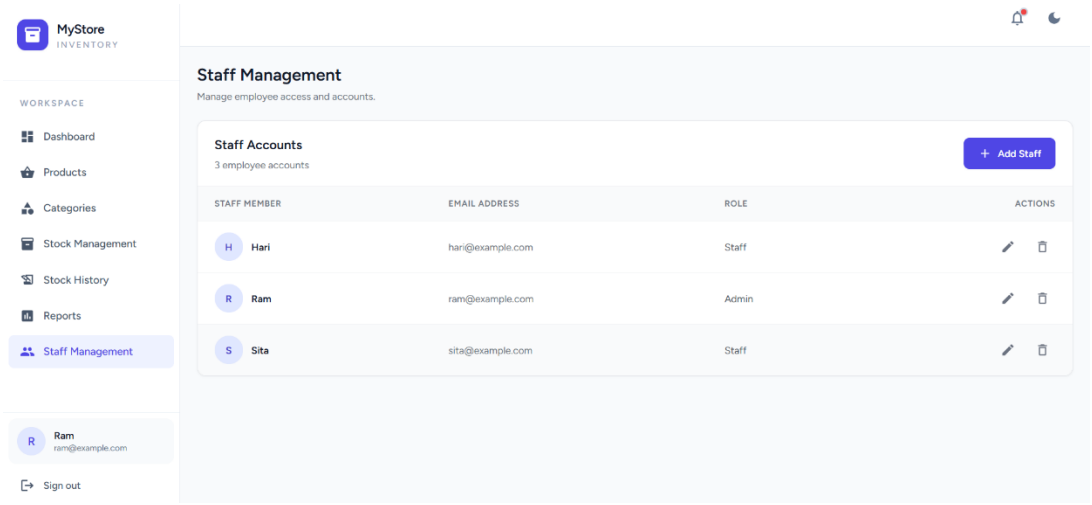
6. Stock History page



7. Reports Page



8. Staff Management page



Implementation and Maintenance

Implementation of the Inventory Management System

Infrastructure setup. The system runs as a web application. A backend handles login, validation and database queries, a MySQL database stores the three tables, and the frontend pages display the data and send form submissions back. Both parts can start on one server and be separated later as usage grows.

Installation and configuration. The database is created from the SQL script in the System Design section, an initial admin account is added, and the backend is configured with the database connection and the rules that decide which role may open which page. The source code is kept under Git version control in the project repository.

Testing and debugging. Before release, each feature is checked with realistic sample data. Tests cover creating and editing products and categories, pagination at 20 records per page, the status rule (in stock, low stock, out of stock), the dashboard totals and role-based access. Dashboard figures are compared with hand-calculated results to confirm that stock totals and inventory value are correct.

User training and documentation. A short guide explains how to log in, add a product, edit a category and read the dashboard. Administrators receive an additional note on managing staff accounts and roles.

Maintenance of the System

- **Bug fixes and updates:** Reported errors are reviewed and fixed promptly, and updates are tested on sample data before they reach the live system.
- **Data management:** Product prices, quantities and minimum stock levels are kept current by staff, and obsolete categories are cleaned up.
- **Backups and security:** The database is backed up regularly, passwords remain hashed, and staff accounts are removed or changed when people leave or change duties.
- **Performance monitoring:** Slow pages and queries are watched as the number of products grows, and indexes or caching are added when needed.
- **Future updates:** The design leaves room for suppliers, purchase orders, stock movement history, reports, low-stock alerts by email and barcode scanning.

Conclusion

The Inventory Management System is designed to replace scattered manual records with a structured database and an organized web-based interface. This case study helped us understand and apply the concepts of Management Information System (MIS) through the analysis and design of a practical system. It covered the identification of problems and objectives, feasibility study, functional and non-functional requirements, data flow diagrams, entity relationship diagram, use case diagram, class diagram, database schema and user interface designs.

The proposed system aims to maintain accurate product and category records, identify low-stock products based on their minimum stock levels, display important inventory information through a dashboard and provide role-based access to different users. Overall, this case study provided us with practical knowledge of how a Management Information System can improve inventory management, data accuracy and decision-making. In the future, the system can be further enhanced by incorporating additional features such as supplier management, purchase orders and stock history to meet the growing needs of the business.